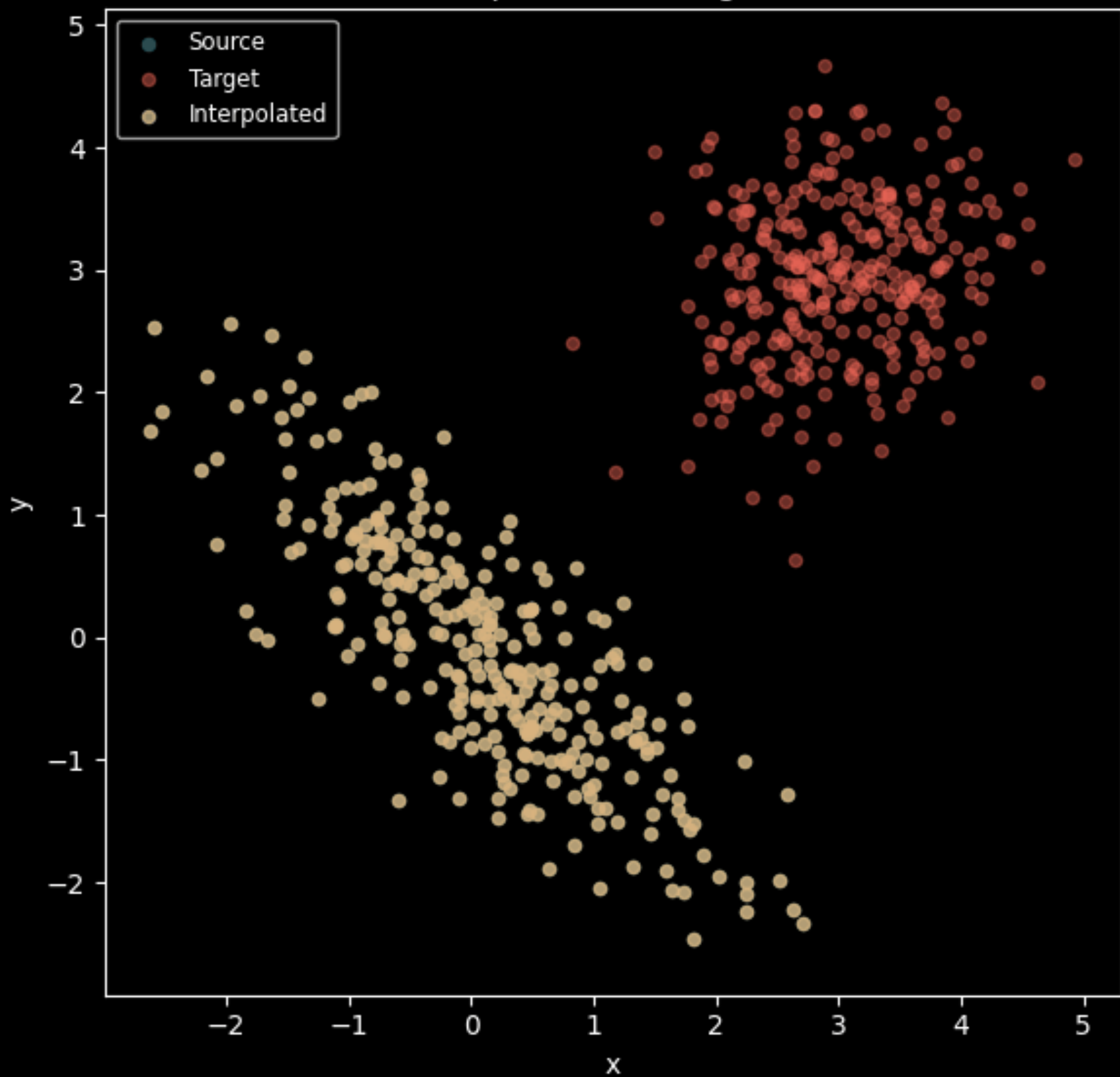


Source $\rightarrow$ Interpolation $\rightarrow$ Target (animation)



```
from scipy.linalg import sqrtm
from numpy.linalg import inv
```

```
def gaussian_ot_map(X,  $\mu_1$ ,  $\mu_2$ ,  $\Sigma_1$ ,  $\Sigma_2$ ):
    """Returns Monge map  $T(X)$  from source to target
    Gaussian under squared cost."""
    sqrt_ $\Sigma_1$  = sqrtm( $\Sigma_1$ )
    inv_sqrt_ $\Sigma_1$  = inv(sqrt_ $\Sigma_1$ )
    A = inv_sqrt_ $\Sigma_1$  @ sqrtm(sqrt_ $\Sigma_1$  @  $\Sigma_2$  @ sqrt_ $\Sigma_1$ ) @ inv_sqrt_ $\Sigma_1$ 
    return  $\mu_2$  + (X -  $\mu_1$ ) @ A.T
```

```
# Sample Gaussians
```

```
 $\mu_1$ ,  $\Sigma_1$  = [0, 0], [[1, -0.8], [-0.8, 1]]
```

```
 $\mu_2$ ,  $\Sigma_2$  = [3, 3], [[0.5, 0.1], [0.1, 0.5]]
```

```
X = np.random.multivariate_normal( $\mu_1$ ,  $\Sigma_1$ , 300)
```

```
Y = np.random.multivariate_normal( $\mu_2$ ,  $\Sigma_2$ , 300)
```

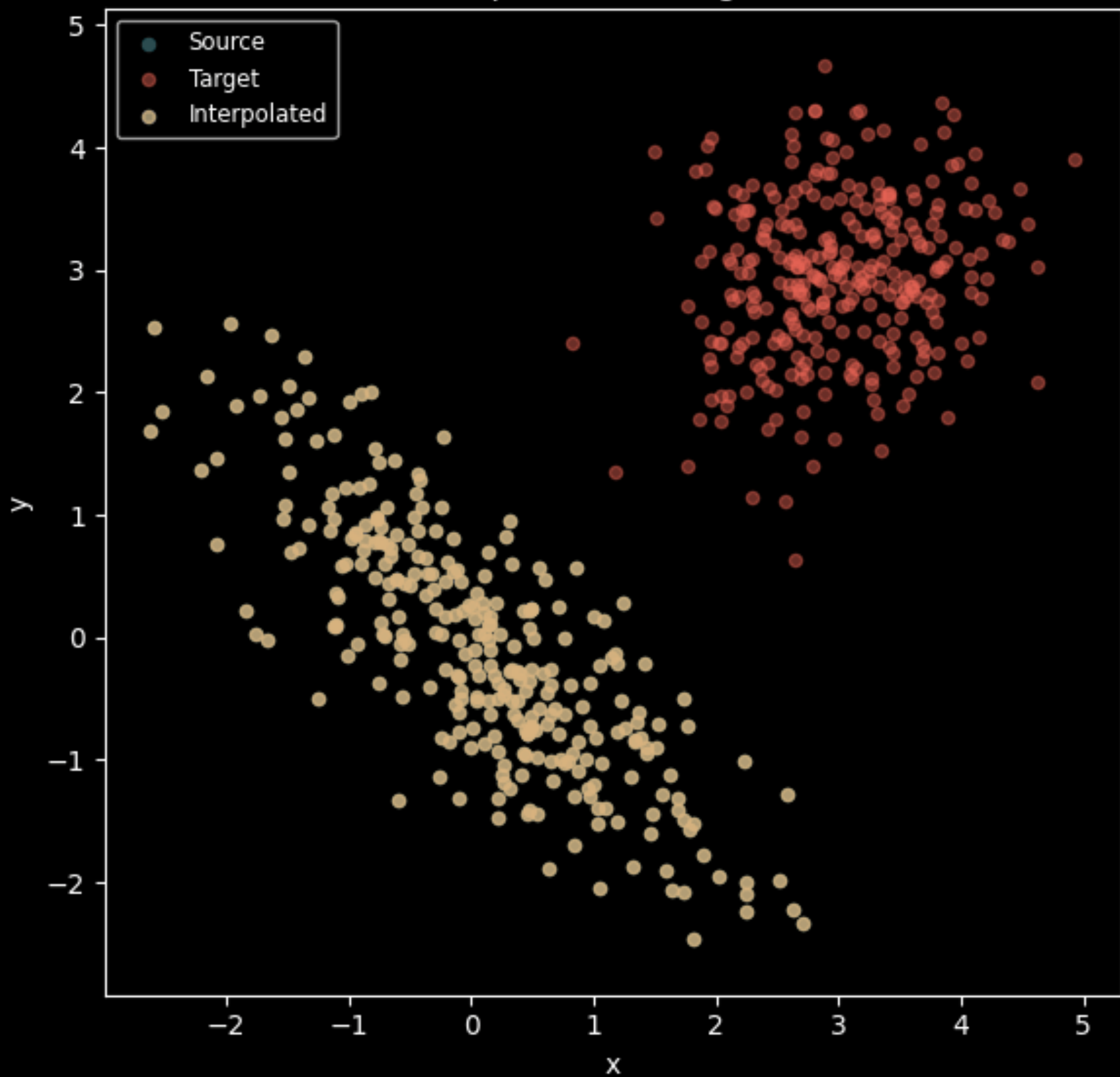
```
TX = gaussian_ot_map(X,  $\mu_1$ ,  $\mu_2$ ,  $\Sigma_1$ ,  $\Sigma_2$ )
```

```
# Interpolation
```

```
alphas = np.linspace(0, 1, 21)
```

```
interpolated = [(1 - a) * X + a * TX for a in alphas]
```

Source $\rightarrow$ Interpolation $\rightarrow$ Target (animation)



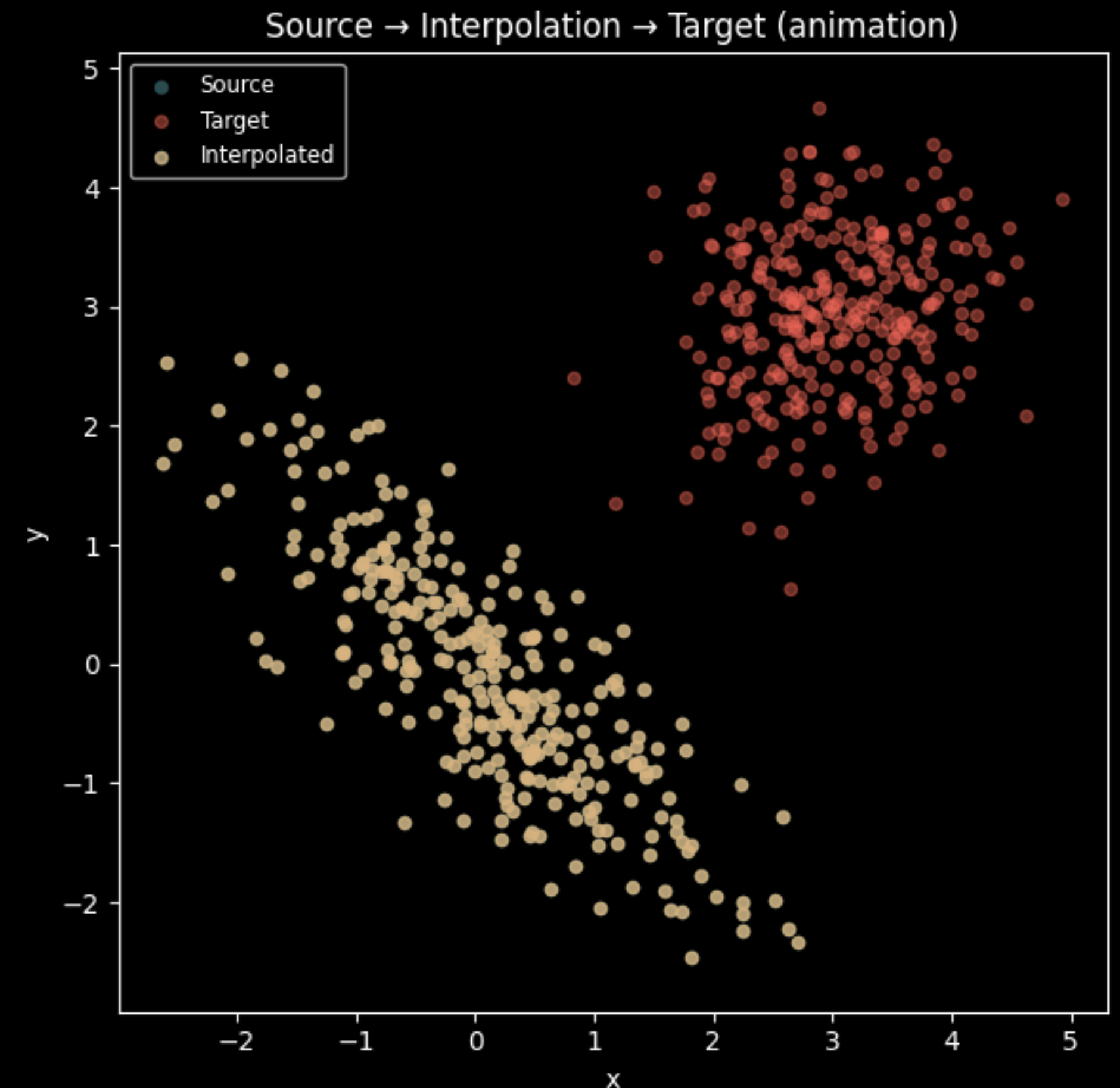
Interactive Demo: Gaussian Monge

```
from scipy.linalg import sqrtm
from numpy.linalg import inv

def gaussian_ot_map(X,  $\mu_1$ ,  $\mu_2$ ,  $\Sigma_1$ ,  $\Sigma_2$ ):
    """Returns Monge map  $T(X)$  from source to target
    Gaussian under squared cost."""
    sqrt_ $\Sigma_1$  = sqrtm( $\Sigma_1$ )
    inv_sqrt_ $\Sigma_1$  = inv(sqrt_ $\Sigma_1$ )
    A = inv_sqrt_ $\Sigma_1$  @ sqrtm(sqrt_ $\Sigma_1$  @  $\Sigma_2$  @ sqrt_ $\Sigma_1$ ) @ inv_sqrt_ $\Sigma_1$ 
    return  $\mu_2$  + (X -  $\mu_1$ ) @ A.T

# Sample Gaussians
 $\mu_1$ ,  $\Sigma_1$  = [0, 0], [[1, -0.8], [-0.8, 1]]
 $\mu_2$ ,  $\Sigma_2$  = [3, 3], [[0.5, 0.1], [0.1, 0.5]]
X = np.random.multivariate_normal( $\mu_1$ ,  $\Sigma_1$ , 300)
Y = np.random.multivariate_normal( $\mu_2$ ,  $\Sigma_2$ , 300)
TX = gaussian_ot_map(X,  $\mu_1$ ,  $\mu_2$ ,  $\Sigma_1$ ,  $\Sigma_2$ )

# Interpolation
alphas = np.linspace(0, 1, 21)
interpolated = [(1 - a) * X + a * TX for a in alphas]
```



Next Time

- Relaxing the problem, Kanotorvich formulation, couplings
- Reading: Peyré & Cuturi (2019), Sections 2 & 3
- Scribing signup released tomorrow